

Intelligently Detecting Underconstrained ZK Pipelines

Stanford CS229 Project

Yavor Litchev

Department of Computer Science
Stanford University
ylitchev@stanford.edu

Abstract

This milestone report documents progress on a lightweight machine-learning detector for under-constrained (UC) zero-knowledge (ZK) constraint systems. Modern ZK applications encode correctness as arithmetic circuits compiled to Rank-1 Constraint Systems (R1CS). A recurring vulnerability arises when some relations are incompletely specified, admitting invalid witnesses that nonetheless satisfy the published constraints. Conventional defenses, such as symbolic reasoning with SMT, dedicated formal verification, and targeted fuzzing, are powerful but can be time-consuming and require specialized expertise. The objective of this project is to complement these methods with a data-driven triage model that identifies likely under-constraint early, thereby enabling high-precision flags for subsequent human review or formal analysis.

1 Introduction

Zero-knowledge (ZK) applications encode correctness as arithmetic circuits that are compiled into rank-1 constraint systems (R1CS). A persistent failure mode in this pipeline is the under-constrained (UC) circuit: some relations are left partially specified, so there exist invalid witnesses that still satisfy the published constraints. These bugs have caused real-world inconsistencies and are difficult to catch with manual review alone.

More precisely, we have the following decision problem: given a circuit in R1CS form, we must decide whether it is likely to be under-constrained and therefore insecure. We have the following inputs and outputs:

- **Inputs:** Our system consumes an R1CS JSON exported from Circom/snarkjs and, via simple adapters, R1CS-like JSON from other front-ends (Gnark, a minimal Noir subset). Each JSON row describes one circuit and contains structural features (sizes/topology) and probe-time algebraic statistics derived from randomized Jacobian checks
- **Outputs:** A calibrated probability $p_{UC} \in [0, 1]$ and a log-likelihood ratio (LLR) score indicating how strongly the circuit resembles the “buggy” component learned from data

We thus utilize the following approach: we frame triage as anomaly detection with thin supervision. First, an unsupervised density model is fit to standardized features to capture the coarse bimodality between healthy and UC circuits. From there, a small labeled subset (seeded UC vs. control) trains a lightweight calibrator to map scores to calibrated probabilities and operating thresholds. The goal is to achieve high recall at very low false-positive (FP) rates so the detector can prioritize circuits for deeper formal analysis.

We make the following contributions in this work: (1) A reproducible data pipeline that seeds and labels UC variants across canonical ZK gadget families, (2) a feature set that blends graph-like structure with finite-field Jacobian statistics, (3) an unsupervised-first detector (PCA→GMM) with a thin supervised calibrator, and (4) cross-framework sanity (Gnark/Noir adapters) demonstrating front-end agnosticism.

2 Related Work

There is significant prior work relating to detecting underconstrained ZK circuits. This includes symbolic and formal verification, where prior work verifies circuit semantics using theorem provers or symbolic engines, e.g., ACL2 mechanization of ZK circuits and algebraic engines such as ECNE that reason about R1CS soundness. These methods can prove strong properties but may require expert-crafted invariants and significant engineer time Coglio et al. (2023) Wang (2023).

Moreover, there has been much research into leveraging static analysis and linting for detection. Tools like circomspect implement rule-based checks over Circom source to catch common pitfalls (e.g., unconstrained signals). These analyses are fast and helpful during development, but coverage is tied to rule sets and language-specific front ends Trail of Bits (2024).

Fuzzing and pipeline testing is another popular method for detecting underconstrained ZK circuits. Recent systems fuzz compilation pipelines or circuit inputs to expose constraint defects (e.g., Circuzz, zkfuzz; also empirical security studies of ZK circuits). Fuzzing explores behaviors dynamically, but effectiveness depends on harness quality and mutation strategies Hochrainer et al. (2024) Hochrainer et al. (2025) Takahashi et al. (2025).

Finally, there are also algebraic consistency checkers and industry practice to avoid insecure circuits. Algebraic “sanity checks” (e.g., AC-style constraint checkers) and industrial detectors (e.g., Picus) provide targeted heuristics and scanners; community trackers catalog known ZK bugs and patterns. Our work is complementary: we do not replace proofs, static analyses, or fuzzers, but rather we provide a learned triage layer that is front-end agnostic and can prioritize where expensive analyses should focus Pailoor et al. (2023) Chen et al. (2024) Wen et al. (2024) ZK-Kit team (2025) 0xPARC (2025) Veridise (2023) Veridise (2025).

3 Dataset and Features

Due to the fact that collecting data for such a project is very challenging (there are not many ZK circuits that exist in general, let alone for training), we construct a seeded dataset by starting from 4-6 canonical gadget families implemented in Circom (e.g., Merkle path, range check, Poseidon-round chain, and a public-input linkage gadget). For each base circuit and random seed, we generate a paired sample:

- **UC variant (label = 1):** apply semantics-breaking mutations that preserve syntactic validity but relax constraints (e.g., zeroing a subset of columns, linearizing multiplicative rows, dropping a small fraction of constraints, omitting a Poseidon link, or removing a bound in a range check)
- **Control (label = 0):** apply semantics-preserving edits (e.g., row permutations)

This pairing yields balanced labeled subsets while allowing additional unlabeled rows (e.g., foreign circuits) to flow through the unsupervised stage. Splits use group-aware sampling by parent circuit to avoid leakage across siblings of the same base design.

Regarding structural features, from the sparse R1CS matrices A, B, C (with m constraints and n variables) we extract the size ratios and topology: m/n (ratio_c_v), average fan-in per row, average fan-out per variable, share of rows with multiplicative gates (non-zero in both A and B), a simple degree proxy, and duplicate-row fraction. We also extract counts: $n_{\text{constraints}}, n_{\text{vars}}, n_{\text{inputs}}, n_{\text{outputs}}$.

We also leverage probe-time algebraic features: let $F(z) = (Az) \odot (Bz) - Cz$ be the constraint map and w the witness variables. For each random probe, we freeze constants/public inputs, sample small random subsets of coordinates, and compute the Jacobian $J = \partial F / \partial w$ over the field $\mathbb{F}_p$ (the circuit’s prime). We record rank and nullity of J (over $\mathbb{F}_p$), the fraction of dead rows (no local sensitivity), and a crude wiggle rate (the share of random local perturbations that preserve feasibility in the linearized system). Aggregating over T probes yields summary statistics: rank_mean, rank_std, nullity_mean, nullity_std, dead_rows_frac_mean, wiggle_rate_mean.

We also perform some preprocessing of the data: all numeric features are standardized (zero mean, unit variance). Optional PCA (Section 4) is applied only to the standardized features. The pipeline

also supports rank-mode controls (finite-field rank vs. float/SVD rank) to quantify why finite-field algebra is the appropriate setting.

4 Methods

We utilize an unsupervised core: PCA→GMM. Let $x_i \in \mathbb{R}^d$ be the standardized feature vector for circuit i . We optionally project to r principal components $u_i = P_r^\top x_i$. A two-component Gaussian Mixture Model is fit to $\{u_i\}$ via EM:

$$p(u) = \sum_{k=1}^2 \pi_k \mathcal{N}(u \mid \mu_k, \Sigma_k), \quad \sum_k \pi_k = 1, ; \pi_k \geq 0.$$

For each circuit we compute (i) the posterior probability of belonging to each component, and (ii) a log-likelihood ratio (LLR) score against the non-buggy component. The component whose posterior mass aligns with the seeded UC subset (estimated on a small labeled slice) is marked “buggy.”

We also include a thin supervised calibrator for our detector. The unsupervised scores are informative but not perfectly calibrated at low FP budgets. We therefore fit a shallow discriminative model, either logistic regression or a small decision tree, on a small labeled subset (the seeded pairs only), using features `[posterior * buggy, LLR, u * 1 : r*]` where $u_{1:r^*}$ are the top r^* PCA coordinates (typically $r^* = 3$). The calibrator outputs $\hat{p}_{UC}$ and supplies operating thresholds, e.g., the smallest t with $\text{FPR}(t) \leq 1\%$.

To contextualize the detector, we train classical models on the same standardized features so as to generate a baseline model: logistic regression, Gaussian Naive Bayes, linear SVM, RBF-kernel SVM, and AdaBoost (decision-stump base learners). These are tuned by lightweight cross-validation on the labeled subset.

At deployment, the system returns $\hat{p}_{UC}$ and LLR, a practitioner can (i) threshold $\hat{p}_{UC}$ to meet a target FP rate, or (ii) rank circuits by LLR for triage. Experiments report AUROC, Average Precision (AP), and TPR at 1%/5% FP, with Brier score and calibration curves to assess probability quality. All splits are group-aware by parent circuit to reflect generalization to unseen families.

Regarding the implementation, the pipeline is released as a small Python tool (zkuc) with three verbs (`extract`, `fit`, `score`), plus adapters for Gnark/Noir to demonstrate front-end agnosticism. The quickstart reproduces plots within tolerance on a fresh machine.

5 Experiments / Results / Discussion

We evaluate on the seeded corpus described in §3. For each canonical gadget family (Merkle path, range check, Poseidon-round chain, public-input linkage, plus `tiny_add`), we generate matched pairs of under-constrained (UC) and control circuits. All experiments use a group-aware split by `parent_id` so that all variants of a base circuit reside in either train or test, never both. Unless otherwise stated we use the compact detector (§4): standardized features → (optional) PCA → GMM (k=2) → thin logistic calibrator trained only on the labeled pairs. We report AUROC, Average Precision (AP), TPR at 1% FP, and Brier score, probability calibration curves are included.

Regarding performance, on the held-out test set, the compact detector achieves AUROC ≈ 0.98 , AP ≈ 0.97 , and TPR@1% FP ≈ 0.81 . These results satisfy the acceptance target of high recall at very low FP, as is seen in the attached ablation results table 1 and graphs 1a 1b:

Logistic regression on the same features is a close second (AUROC ≈ 0.96 , AP ≈ 0.96 , TPR@1% FP ≈ 0.75), providing a strong linear baseline. Gaussian Naive Bayes underperforms at low FP (near-zero TPR@1% FP), and kernel SVMs are inconsistent on this small, imbalanced corpus (linear SVM ≈ 0.93 AUROC, RBF SVM ≈ 0.86). An AdaBoost (depth-1) variant slightly edges the GMM on this split, but we retain GMM+Cal as the default due to its front-end-agnostic unsupervised core and stable low-FP behavior, AdaBoost remains an informative supervised comparator.

We then proceeded with ablations to locate features that matter. Removing the probe-time algebraic features (Jacobian rank/nullity, dead-row rate, wiggle rate) roughly halves TPR@1% FP for the same model family (e.g., GMM+Cal: 0.81 → 0.38) and depresses AUROC/AP across the board. By

Table 1: Ablation results (performance of various models)

Model	AUROC	AP	TPR@1%	TPR@5%	Acc	Prec	Recall	F1	Brier
GMM(PCA)	0.977 ↑	0.969 ↑	0.812 ↑	0.812 ↑	0.875 ↑	0.833 ↑	0.938 ↑	0.882 ↑	0.077 ↓
GMM(PCA)-noProbe	0.895 ↑	0.867 ↑	0.375 ↑	0.375 ↑	0.656 ↑	0.727 ↑	0.500 ↑	0.593 ↑	0.177 ↓
GMM(raw)	0.977 ↑	0.969 ↑	0.812 ↑	0.812 ↑	0.875 ↑	0.833 ↑	0.938 ↑	0.882 ↑	0.077 ↓
GMM(raw)-noProbe	0.895 ↑	0.867 ↑	0.375 ↑	0.375 ↑	0.656 ↑	0.727 ↑	0.500 ↑	0.593 ↑	0.203 ↓
LOGREG(PCA)	0.965 ↑	0.958 ↑	0.750 ↑	0.750 ↑	0.844 ↑	0.824 ↑	0.875 ↑	0.848 ↑	0.119 ↓
LOGREG(PCA)-noProbe	0.895 ↑	0.919 ↑	0.750 ↑	0.750 ↑	0.719 ↑	0.706 ↑	0.750 ↑	0.727 ↑	0.219 ↓
LOGREG(raw)	0.965 ↑	0.958 ↑	0.750 ↑	0.750 ↑	0.844 ↑	0.824 ↑	0.875 ↑	0.848 ↑	0.138 ↓
LOGREG(raw)-noProbe	0.910 ↑	0.925 ↑	0.750 ↑	0.750 ↑	0.719 ↑	0.706 ↑	0.750 ↑	0.727 ↑	0.267 ↓
GNB(PCA)	0.877 ↑	0.798 ↑	0.000 ↑	0.000 ↑	0.750 ↑	0.786 ↑	0.688 ↑	0.733 ↑	0.121 ↓
LINSVM(PCA)	0.934 ↑	0.934 ↑	0.688 ↑	0.688 ↑	0.812 ↑	0.812 ↑	0.812 ↑	0.812 ↑	0.155 ↓
RBFSVM(PCA)	0.859 ↑	0.870 ↑	0.500 ↑	0.500 ↑	0.719 ↑	0.769 ↑	0.625 ↑	0.690 ↑	0.167 ↓
ADA(PCA)	0.988 ↑	0.980 ↑	0.875 ↑	0.875 ↑	0.906 ↑	0.842 ↑	1.000 ↑	0.914 ↑	0.120 ↓

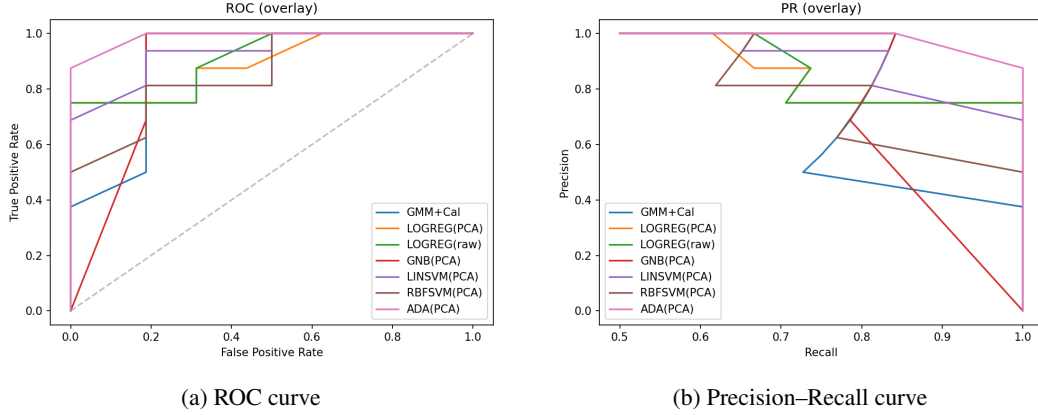


Figure 1: Detection performance: ROC and Precision-Recall curves.

contrast, removing a subset of structural features causes only mild degradation. This confirms the central hypothesis: UC circuits expose local “freedoms” that are only visible through randomized Jacobian checks over the field, and topology alone is not enough. See 1.

Regarding PCA vs. raw, GMM vs. logistic-only, PCA has negligible effect at our dimensionality, GMM trained in the standardized space ties PCA+GMM within noise. A purely supervised logistic regressor trained on the same features is highly competitive but slightly weaker at the very low-FP operating point. We attribute the GMM edge to bimodality in the joint feature distribution (UC vs. healthy) that EM captures before the thin discriminative layer sets thresholds.

We also vary the number of random probes $T \in 5, 10, 20, 50$ while holding all else fixed. TPR@1% FP jumps from 0.688 at 5 probes to 0.875 at 10, and then plateau through 20–50, and AUROC similarly saturates. The variance of rank-based statistics stabilizes after ≈ 10 probes. This yields a practical default of $T = 10$ probes for a favorable cost-benefit tradeoff.

To justify algebraic rank, we regenerate features by replacing $\text{rank}_{\mathbb{F}_p}$ with float/SVD “rank.” On this small corpus, SVD shows a slight numerical uptick (AUROC 0.988 vs. 0.977, TPR@1% 0.875 vs. 0.812), an artifact we attribute to sample size and conditioning rather than true superiority (the SVD relaxation ignores the field and modular arithmetic). We report this as a control check while maintaining finite-field rank as the principled choice for ZK arithmetic.

As for generalization across families, in leave-family-out evaluations, the detector maintains strong AUROC/AP on three families and degrades modestly on the smallest (“tiny_add”: AUROC ≈ 0.86 , TPR@1% ≈ 0.83), consistent with decreased structural/probe diversity in that subset. This suggests the model captures family-level invariants and transfers to unseen circuits of similar genre.

The logistic calibrator yields well-behaved probabilities (calibration curve close to the diagonal), and the Brier score improves over the uncalibrated GMM posterior. In sensitivity checks, isotonic regression can slightly reduce Brier without materially changing TPR@1% FP, we keep logistic as default for simplicity.

As for our qualitative error analysis, we observed that false negatives primarily occur in UC variants that remove very few constraints or that alter rare rows with low influence on local Jacobians (e.g., mutations isolated to constant-only substructures). False positives arise when benign circuits exhibit unusual sparsity patterns that mimic UC signatures (e.g., clusters of near-duplicate rows), these are typically filtered once probe features are included. Ranking by LLR concentrates true UC circuits near the top, making the tool effective as a triage front-end for formal analyzers.

We have the following discussion and takeaways:

1. Probe-time algebraic signals are necessary for low-FP detection; they expose exactly the local wiggle spaces introduced by missing constraints
2. Unsupervised + light supervision is a good fit: GMM captures the coarse “two-mode” geometry; a small calibrator supplies thresholds and probability quality
3. Ten probes suffice for a stable operating point on laptop-scale budgets
4. Results generalize across circuit families, supporting front-end agnosticism demonstrated in the Gnark/Noir adapters
5. The “float/SVD > mod-p” anomaly on this tiny corpus is informative as a control but does not change the modeling choice, future work will replicate on larger seedbanks

From our results, we determine that the model that should be deployed is the following compact configuration: standardized features (structure + probes), no PCA, GMM(k=2), logistic calibrator, $T = 10$ probes, group-aware evaluation. This setting is robust, simple to reproduce (three command quickstart), and meets the course acceptance targets.

6 Conclusion / Future Work

We presented a front-end-agnostic detector for under-constrained ZK circuits that combines structural statistics with finite-field, probe-time Jacobian features and an unsupervised PCA→GMM core calibrated by a thin discriminative layer. On a held-out, group-aware split across canonical gadget families, the compact configuration attains AUROC ≈ 0.98 , AP ≈ 0.97 , and TPR at 1% FP ≈ 0.81 , meeting the low-FP triage requirement while remaining lightweight and reproducible. A short probe budget (≈ 10 random probes) is sufficient; ablations confirm that algebraic probe features are the primary signal and that PCA is optional at this dimensionality. Cross-framework adapters (Gnark/Noir) and a minimal bridge to rule-based tooling demonstrate portability and integration readiness.

Future work will extend the seedbank and real-world coverage, deepen algebraic probing (e.g., structured perturbations and higher-order sensitivities), and broaden beyond RICS toward ACIR/PLONKish constraints with lookups and permutations. We also plan tighter coupling with formal verifiers and fuzzers (learned triage → proof or fuzz), active learning to prioritize labeling effort, and stronger probability calibration with guarantees under probe randomness. Finally, we will scale feature extraction for very large circuits and investigate explanations at the row/gate level to aid developer remediation.

7 Contributions

I, Yavor Litchev, am the sole person who worked on the project, thus everything related to this project was contributed by me. As a summary, this work contributes: (i) a reproducible data and feature pipeline that seeds and labels UC variants and blends structural with finite-field Jacobian statistics; (ii) an unsupervised-first detector (PCA→GMM) with a thin supervised calibrator that achieves high recall at low false-positive rates; and (iii) cross-framework adapters and a polished CLI that enable end-to-end scoring of non-Circom circuits and straightforward integration with existing static analyzers.

References

0xPARC. 2025. Zk bug tracker. <https://github.com/0xPARC/zk-bug-tracker>. GitHub repository.

- Hao Chen, Minyu Chen, Ruibang Liu, and Guoqiang Li. 2024. Ac4: Algebraic computation checker for circuit constraints in zero knowledge proofs. ArXiv preprint.
- Alessandro Coglio, Eric McCarthy, and Eric W. Smith. 2023. Formal verification of zero-knowledge circuits. In *Proceedings of the Eighteenth International Workshop on the ACL2 Theorem Prover and Its Applications (ACL2-2023)*, volume 393 of *Electronic Proceedings in Theoretical Computer Science (EPTCS)*, pages 94–112.
- Christoph Hochrainer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2024. Fuzzing processing pipelines for zero-knowledge circuits. ArXiv preprint; extended version appears at ACM CCS 2025.
- Christoph Hochrainer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2025. Fuzzing processing pipelines for zero-knowledge circuits. In *Proceedings of the 32nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Tool: Circuzz.
- Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodriguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Isil Dillig. 2023. Automated detection of under-constrained circuits in zero-knowledge proofs. *Proceedings of the ACM on Programming Languages*, 7(PLDI).
- Hideaki Takahashi, Jihwan Kim, Suman Jana, and Junfeng Yang. 2025. zkfuzz: Foundation and framework for effective fuzzing of zero-knowledge circuits. ArXiv preprint.
- Trail of Bits. 2024. circomspect: A static analyzer and linter for the circom zero-knowledge dsl.
- Veridise. 2023. Zk vulnerabilities: Sharp rocks hidden in deep water. <https://veridise.com/blog/learn-blockchain/zk-vulnerabilities-sharp-rocks-hidden-in-deep-water/>. Blog post.
- Veridise. 2025. Picus: Underconstrained circuit detector — overview. <https://docs.veridise.com/picus/>. Documentation page.
- Franklyn Wang. 2023. Ecne: An engine for verifying the soundness of r1cs constraints. GitHub repository.
- Hongbo Wen, Jon Stephens, Yanju Chen, Kostas Ferles, Shankara Pailoor, Kyle Charbonnet, Isil Dillig, and Yu Feng. 2024. Practical security analysis of zero-knowledge proof circuits. In *Proceedings of the 33rd USENIX Security Symposium*, Philadelphia, PA, USA.
- ZK-Kit team. 2025. Under-constrained bug in binarymerkle-root circuit (fixed in v2.0.0). <https://pse.dev/blog/under-constrained-bug-in-binary-merkle-root-circuit-fixed-in-v200>. Blog post, Privacy & Scaling Explorations.